UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

DEPARTAMENTO ACADÉMICO DE INGENIERÍA INFORMÁTICA

REDES DE COMPUTADORAS 2

Lab 01: Programación de Sockets en TCP con Python

Leunam

"Dime y lo olvidaré. Muéstrame y lo recordaré. Involúcrame y lo entenderé"

Proverbio chino

---

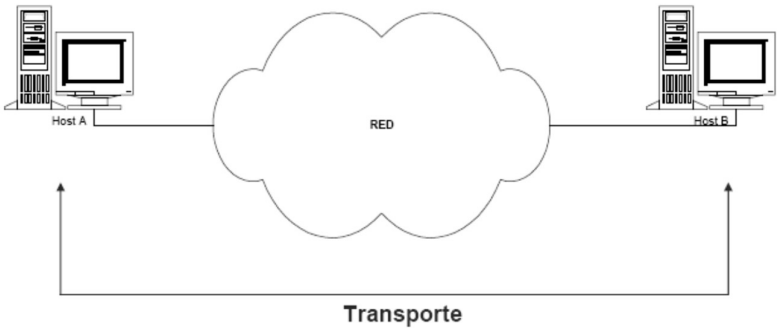
1. OBJETIVOS:

3.1. Entender lo básico de la comunicación por redes.
3.2. Programar aplicaciones de red usando sockets.
3.3. Aprender cómo construir aplicaciones cliente/servidor que se comunican usando sockets
3.4. Aprender el uso de sockets en Python.

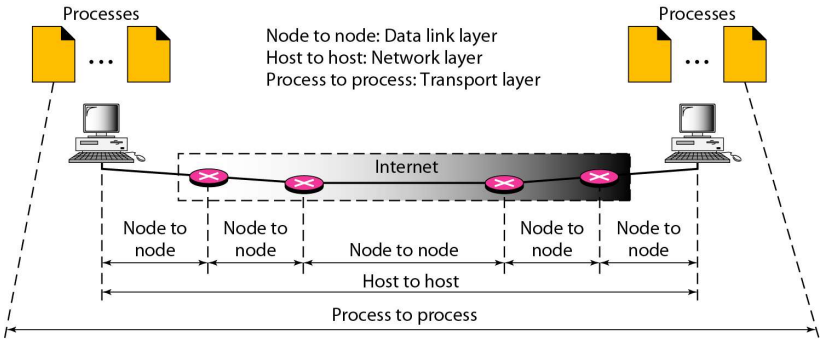
2. BASE TEORICA COMPLENTARIA:

Capa de Transporte:

– Puede ofrecer a sus usuarios un sistema transparente de intercambio de datos confiable, operando de extremo a extremo (end-to-end).



– La capa de transporte es la responsable por la entrega proceso-a-proceso – la entrega de un paquete, o la parte de un mensaje, desde un proceso a otro.

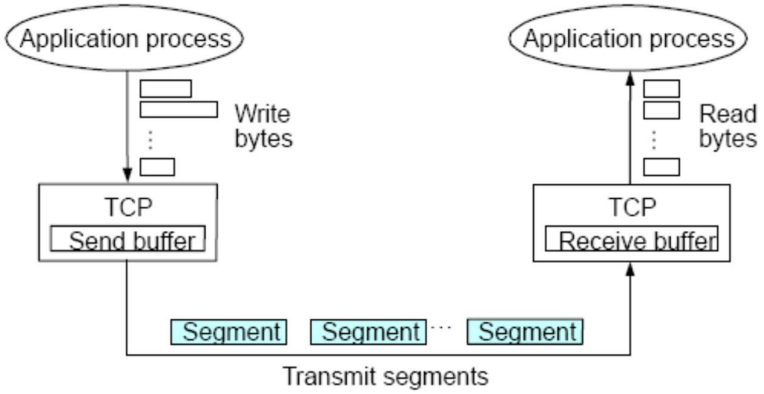


TCP: Transmission Control Protocol

– RFC 9293, 2022, Categoría: Standards Track.
– Transporte/Servicio orientado a la conexión.
– Protocolo de transporte de Internet.
– Transferencia de datos full-dúplex
– Transporte confiable:
– Control:
  - de transmisión
  - de flujo

- de errores
- de congestión de red
- ...

– Provee servicio a la capa de aplicación.
– Transferencia de un stream de bytes entre sistemas finales (end-to-end).
– …



– Cuando 2 aplicaciones desean comunicarse el uno al otro confiablemente, ellos establecen una conexión y envían datos de una parte a la otra sobre esa conexión.
– TCP provee un canal punto-a-punto para aplicaciones que requieren comunicaciones confiables.
  - HTTP (HyperText Transfer Protocol), FTP (File Transfer Protocol), y Telnet son algunos ejemplos de aplicaciones que requieren un canal de comunicación confiable.
  - El orden en la cual los datos son enviados y recibidos sobre la red es crítico para el éxito de estas aplicaciones.
  - Cuando HTTP es usado para leer desde un URL, los datos tienen que ser recibidos en el orden en el cual fueron enviados. De otra manera, se acabaría con un archivo HTML revuelto, un archivo ZIP corrupto, o alguna otra información inválida.

Puertos:

– En general, una computadora tiene una simple conexión física a la red.
  - Todos los datos destinados a una computadora en particular llegan a través de esa conexión.
  - Sin embargo, los datos pueden ser pretendidos por diferentes aplicaciones corriendo en el mismo computador.
  - ¿Así como la computadora sabe a cuál aplicación reexpedir los datos?

A través del uso de los puertos

– Cada proceso que desea comunicarse con otro proceso se identifica a sí mismo para la suite de protocolos TCP/IP por 1 o más puertos.
– Los datos trasmitidos por el Internet están acompañados por información de direccionamiento que identifica a la computadora y al puerto para el cual están destinados.
  - La computadora está identificada por su dirección IP de 32-bit.
  - Los puertos son identificados por un número de 16-bit, los cuales TCP y UDP usan para entregar los datos a la aplicación correcta.

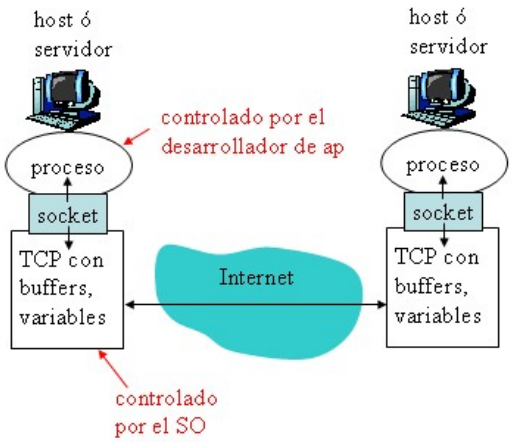
– Hay 2 rangos de puertos:
  - Puertos "Well-known" o bien-conocidos:
    - 0: reservado
    - 1 – 1023: usado por procesos servidor
  - Puertos Efímeros:
    - 1024 - 65535: usado por procesos cliente.

– Sin embargo, el IETF IANA define 3 rangos de puertos:
  - Puertos Well-Known (bien-conocidos) o Puertos de Sistema.
    - rango: 0 – 1023
    - son asignados por el IANA
  - Puertos Registrados o Puertos de Usuario.
    - rango: 1024 – 49151
    - no deberían ser usados sin el registro del IANA.
  - Puertos Dinámicos o Puertos Privados o Puertos efímeros.
    - rango: 49152 – 65535
    - típicamente usados como puerto fuente por clientes TCP o UDP.

Socket:

– Una interface:
  - en el host local.
  - creada por la aplicación.
  - controlada por el SO

(Una estructura de datos) dentro de la cual el proceso de aplicación puede tanto enviar como recibir mensajes a/desde otro proceso de aplicación.



– El par <dirección IP>, <Nº de puerto> brinda una identificación única para los servicios de la Capa de Aplicación en un host, y sirve para identificar a un socket.
– Cada lado de una conexión TCP tiene un socket que puede ser identificado por:
  - la 3-tupla <TCP, dirección IP, # puerto>
    o
  - la 2-tupla <dirección IP, # puerto>.
– Si 2 procesos están comunicándose sobre TCP, ellos tienen una conexión lógica que es de una manera única identificable por los 2 sockets involucrados, esto es, por:
  - una 5-tupla <TCP, dirección IP local, # puerto local, dirección IP remota, # puerto remoto>
    o
  - una 4-tupla <dirección IP local, # puerto local, dirección IP remota, # puerto remoto>.
– Entonces, una conexión TCP está identificada por un par de sockets (el del cliente y el del servidor) en ambos extremos.



3. RECURSOS:

3.1. Python.

3.2. Windows / Ubuntu.

4. DESARROLLO DE LA PRACTICA:

En el presente trabajo de laboratorio se visualizará el estado de las conexiones TCP y se implementará tanto un cliente que envía datos en minúscula a un servidor, como un servidor que recibe datos en minúscula desde un cliente y lo devuelve de regreso, pero en mayúsculas haciendo uso del lenguaje Python.

4.1. Estado de las conexiones TCP:

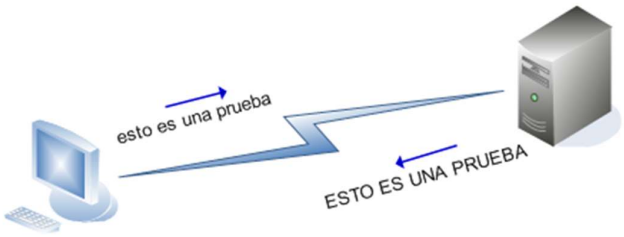
- Abrir una ventana de Terminal o de CLI (Interface de Línea de Comandos).
- (Investigar las opciones de netstat).
- En el Prompt de comandos tipear:

netstat -an

Conexiones activas

Proto	Dirección local	Dirección remota	Estado
TCP	0.0.0.0:80	0.0.0.0:0	LISTENING
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING
TCP	127.0.0.1:53	0.0.0.0:0	LISTENING
TCP	127.0.0.1:23401	0.0.0.0:0	LISTENING
TCP	192.168.1.6:53	0.0.0.0:0	LISTENING
TCP	192.168.1.6:139	0.0.0.0:0	LISTENING
TCP	192.168.1.6:6419	69.172.201.208:80	ESTABLISHED
TCP	192.168.1.6:6420	16.58.219.130:443	ESTABLISHED
UDP	192.168.56.1:138	*:*	
UDP	192.168.56.1:1900	*:*	
UDP	192.168.56.1:56284	*:*	

4.2. Escribir una aplicación Cliente-Servidor que implemente el servicio “CAMBIAR MENSAJE EN MINUSCULAS A MENSAJE EN MAYUSCULAS”.



- En la programación de sockets, la estructura del programa es en mucho el mismo:

Abrir un socket.

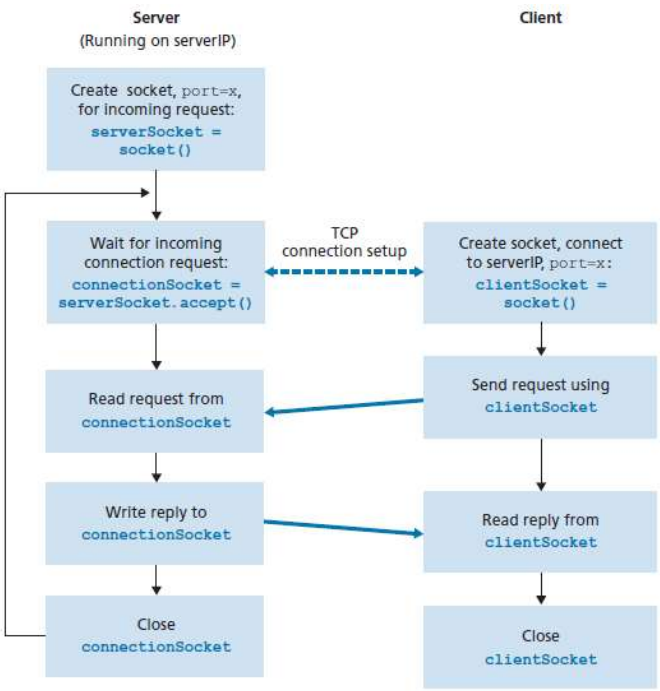
Abrir un stream de entrada y un stream de salida para el socket.

Leer desde y escribir hacia el stream conforme al protocolo del servidor.

Cerrar los streams.

Cerrar el socket.

Aplicación Cliente-Servidor usando TCP:



– Editar los siguientes archivos Python.

APLICACIÓN CLIENTE:

ClienteTCP.py

```
# ClienteTCP.py

from socket import *

# inicializar variables
idServidor = 'dirección_IP'
#idServidor = '192.168.1.36'

puertoServidor = 65432

# crear socket TCP para comunicarse con el servidor,
#   en el puerto remoto 65432
socketCliente = socket(AF_INET, SOCK_STREAM)

# iniciar conexión TCP hacia el servidor, indicando la dirección del servidor
socketCliente.connect((idServidor, puertoServidor))

# obtener alguna entrada desde el teclado del usuario
mensaje = input('Entrar una sentencia en minúsculas:')

# enviar el string a traves del socket del cliente hacia la conexión TCP
socketCliente.send(mensaje.encode())

# ¡el cliente se queda en espera!

# leer desde su socket, los datos son colocados en el string mensajeModificado
mensajeModificado = socketCliente.recv(1024)
```

```
# imprimir string recibido
print ('Desde el servidor:', mensajeModificado.decode())

# cerrar socket
socketCliente.close()
```

APLICACIÓN SERVIDOR:

ServidorTCP.py

```
# ServidorTCP.py

from socket import *

# inicializar variables
puertoServidor = 65432

# crear socket de escucha (o de bienvenida)
socketServidor = socket(AF_INET,SOCK_STREAM)

# asociar nro. puerto del servidor con el socket
socketServidor.bind(('',puertoServidor))

# servidor se queda escuchando por peticiones de conexión TCP
#   (desde el cliente, con un máximo de conexiones en cola de 2)
socketServidor.listen(2)

print ('El servidor está listo para escuchar')

# lazo 'eterno'
while True:
        # el servidor espera en el metodo accept() por peticiones entrantes,
        #   un nuevo socket dedicado para este cliene en particular es
        #   creado al retorno de este metodo
        socketConexion, addr = socketServidor.accept()

        # leer solo los bytes desde el socket
        sentencia = socketConexion.recv(1024).decode()

        # capitalizar la sentencia
        sentenciaCapitalizada = sentencia.upper()

        # enviar sentencia modificada de regreso al cliente
        socketConexion.send(sentenciaCapitalizada.encode())

        # cerrar la conexión a este cliente (el socket de escucha sigue abierto)
        socketConexion.close()
```

– En Windows/Ubuntu abrir 2 ventanas de CLI (Interface de Línea de Comandos) o de Terminal, y en cada ventana cambiarse al directorio donde están los archivos Python.

  ServidorTCP.py
  ClienteTCP.py

– En Windows/Ubuntu, en la ventana del servidor ejecutar el programa Servidor con tipear lo siguiente:

Windows:
python ServidorTCP.py
o
py ServidorTCP.py

Ubuntu:
python3 ServidorTCP.py

– En Windows/Ubuntu, en la ventana del cliente ejecutar el programa Cliente con tipear lo siguiente:
Windows:
python ClienteTCP.py
o
py ClienteTCP.py

Ubuntu:
python3 ClienteTCP.py

5. EJERCICIOS PROPUESTOS:

5.1. Completar el comando dado a continuación, para que SOLAMENTE muestre las transmisiones/conexiones asociadas con un único Número de Puerto ESPECÍFICO, es decir, para que en la columna "Dirección Local" o en la columna "Dirección Remota / Peer" aparezcan solo las conexiones asociadas a un Número de Puerto ESPECÍFICO.
Nota: En Windows o en Linux se verificará en el mismo laboratorio la validez de la respuesta para esta pregunta.

En Windows:
netstat -aon _____

o

En Linux:
ss -tuapn _____

Tipo de trabajo	Individual
Tipo de informe	Digital: PDF
Fecha de entrega	dd/mm/aaaa